In-class Activity

EK 505 - Prof. Tron

2026-09-21

For questions tagged with the label **scan**, answer all the questions in the space provided; to submit, scan your work and submit to Gradescope. Please make sure to upload the pages in the correct order. The question tagged with the label **image** require you need to submit an image on Gradescope for evaluation.

Problem 1: Differential of the forward kinematics of a spherical robot

From a previous in-class activity, the *Forward Kinematic* of the spherical robot shown in Figure 1 is given by the 4×4 homogeneous transformation

$$ {}^{\mathcal{B}_0}T_{\mathcal{E}} = T_{p_z}(\ell_{\text{red}})T_{R_z}(q_0)T_{p_z}(\ell_{\text{green}})T_{R_x}(q_1)T_{p_z}(\ell_{\text{blue}}) \tag{1} $$

Question scan 1.1. Given ${}^{\mathcal{B}_0}T_{\mathcal{E}}$, how do you compute the translation of the end effector ${}^{\mathcal{B}_0}p_{\mathcal{E}}$ in the base link frame $\mathcal{B}_0$?

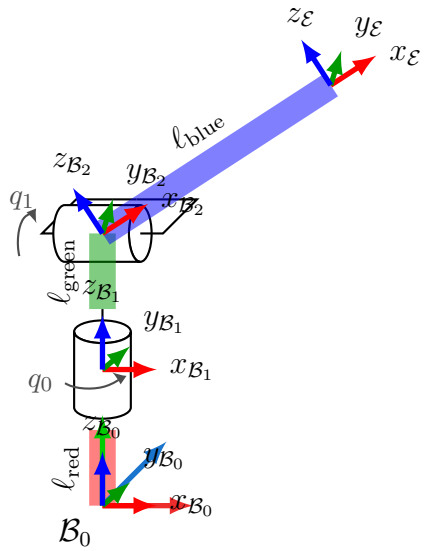


Figure 1: Kinematic chain of a simplified spherical robot

Question scan **1.2.** Assume that $\ell_{\text{red}} = \ell_{\text{green}} = 0$ and $\ell_{\text{blue}} = 0.8$. Using the expressions above and skew-symmetric matrices (written using the *hat map*), give an expression for the linear velocity ${}^{\mathcal{B}_0}\dot{p}_{\mathcal{E}}$ in the base link frame $\mathcal{B}_0$.

Question image **1.1.** Use the `spatialmath` toolbox and NumPy to implement Question scan 1.2 in the function `robot_fkine_ee_jacob( q , qd )` in the provided file `ek505_activity3.py`. The variable `qd` represents $\dot{q}$.

Then follow the other TODO items in the same file to compute the Jacobian of the end effector position by following the steps below.

🔎 The Jacobian $J(q)$ is the joint-dependent matrix mapping joint velocities to end effector velocities, i.e., ${}^{\mathcal{B}_0}\dot{p}_{\mathcal{E}} = J(q)\dot{q}$.

1) Create the symbolic variable `q` via SymPy to represent the joint coordinates $\left[\begin{smallmatrix} q_0 \\ q_1 \end{smallmatrix}\right]$.

2) Compute the symbolic expression of the position of the end effector by using the Robotics Toolbox's forward kinematic function with `q` as the input, and then obtain the Jacobian by applying SymPy's `.jacobian(_)` method.

🔎 You will need to convert from NumPy arrays to SymPy matrices.

3) Use your function `robot_fkine_ee_jacob(_)` to compute the Jacobian, using `q` as the input; `qd` needs to be selected to compute one column of the Jacobian at a time.

🔎 Recall that given a matrix J, the expression Je_i (where e_i is the i-th element of the standard basis) is equivalent to the i-th column of J.

4) Compute the Jacobian using the Robotics Toolbox's built-in method `.jacob0(_)`.

🔎 You will need to use SymPy's `simplify(_)` function to obtain a simpler expression.

Run the provided `main(_)` function, which prints the three different expressions of the Jacobian: they should coincide. Submit a picture of both the code and of output.

Question scan **1.3.** What is the meaning of the extra three rows in the output of the `.jacob0(_)` method?